

model is. So, there is a large market for getting the right data.

The second part is building a model. There are many applications that do not require creating a model like face recognition and object detection. Because these common applications already have ready models available out there—one just needs to choose the right one for the application. And after that, you need to further train your model, as most models are usually trained for different sets of data which do not give a good output.

As each and every library is good at one thing or the other, one needs to carefully choose the right one that is suitable for the application.

After having a working model, it must be used purposefully to generate enough money. Some of the questions which the money-making guys should focus on are: What should it (the model) be used for? What is the market looking for?

Apart from all this, there is a major competition since everyone in AI is working directly on the cloud with respect to model training and data. However, AI can run equally well on embedded computers. This is commonly unknown to people.

All AI will happen on the edge

When running your model on the cloud, usually Google engine API or Amazon's object recognition models are used for which certain amount is paid. Once the solution is built, over a period of time you have to pay a lot of money.

Instead, the same model can be coded onto hardware. On the embedded system you can get the performance of the cloud without having to pay on a long term basis. All analysis happens on the embedded device and it sends only limited data to the cloud.

Edge analytics is going to be the next big thing for AI. All of the data which the AI library is sending down will have computation done by the processor and give results. Example, in the past, when gaming came in, all the mathematical computation was done in GPU. Now, there are vector processing units on the chips which can process vectors (data) coming from the cloud very fast.

The GPU is also good at doing floating-point calculations.

People are unaware that a lot of AI applications can be built on a chip which the companies can optimise.

The number of libraries available for AI today stands at approximately 800 plus. And this is also the amount of competition in the AI space. This might sound unbelievable, but when computers came in, people never imagined it to turn out to be this big. Gradually, it became a necessity in our lives. And AI is going to be much bigger than that. In the next five years, AI is going to be inside everything. A lot of products will come with AI built into them.

Currently, there are twenty plus companies dealing with AI technology. Some of the popular ones are Tencent, TensorFlow, Caffe, Chainer, ONNX and PyTorch.

Challenges faced in embedded

Challenges such as model size, choosing the appropriate model and framework are commonly faced by people who work on AI. The system side challenges are something that most people are not aware of. These include:

Platform. In the cloud, the code is pre-installed. But for embedded, you need to fetch the source and compile it into the machine.

Fetching. Finding the right source is also a challenge. One needs to do a lot of research to find the right source, the correct patches and configure them.

Architecture support. Mostly ARM is used, which can be either ARM7 or ARM8. With the help of Neon support in ARM8, hardware FPUs (floating point units) provide 5x performance. It is very advantageous but challenging as well.

Compilation. Cross-compiling does not give the required performance. So, to derive performance and make a library work optimally on hardware, you need to do a native compilation. This requires a very long duration of coding and consumes a lot of time.

Installation. You may encounter several error messages after compilation. Care needs to be taken to install libraries correctly, especially for new machines.

Not just these, all libraries usually

have some dependencies. Dependencies are some additional packages that are needed to run a library. Each tool/framework/library on an average has 4-5 dependencies.

Those packages also need to be compiled to get optimal performance natively from the AI libraries installed on your embedded system. Software called *Docker* is the quickest way to run packages, but it is not optimised while it is running and must be used with care.

There is this perception in the industry that to run AI on the edge, you need a lot of computation or one needs to have better computing power so that enough performance is obtained. However, this is not correct. When that solution is sold, your cost is going to be higher. So, the focus should be on better engineering, which makes the price come down. This should happen without compromising on the latest technological advances.

OpenGL and OpenCL libraries

In an embedded system, while a CPU will be obviously present, GPU and NPU are optional. If you have GPU, then it will have OpenGL or OpenCL library. OpenGL is a graphics library which passes on to GPU the graphic calculations it receives. OpenCL is a compute library which checks the amount of computational power on the hardware; whatever computational request it gets is distributed among the system. By adding OpenCL, your performance goes very high. It is an additional layer which handles requests much better than OS.

There is still a lot to be done on the application side. After writing the code in the cloud, when you run it, the analysis process begins. In this process, the code generates an unknown number of threats which can cause hardware failure if the system cannot handle them. But in embedded, it is custom.

Here, edge computing plays a major role by taking computing actions directly on the hardware instead of sending data to the cloud for computation.

As embedded AI space is coming up, there are some frameworks which are getting built only for this side. Currently, TVM is doing the best in this area. **EFY**